

INDIAN CYBER CRIME COORDINATION CENTRE, NEW DELHI

MINISTRY OF HOME AFFAIRS
2024-25



INTERNSHIP REPORT

Submitted by Vanshita Arya

Dec 24 – Feb 25

Sneha Singh Bhaskar
Technical Assistance

Amit Sankholia
Deputy Director

Lt. Col. Badal Kaushik
Director, CFMC

Lt Col Arvind Kumar
Director, NCTC(I4C)

ACKNOWLEDGMENT

I would like to thank my supervisors and mentors at MHA I4C, particularly Director, CFMC, **Lt. Col. Badal Kaushik**, and **Mrs. Sneha Singh Bhaskar**, for their tremendous guidance, support, and encouragement throughout my internship. Their knowledge and insights have greatly influenced my understanding of cyber fraud detection, financial crime analysis, and fraud mitigation measures.

A particular thank you to my team members and colleagues, whose collaboration and shared knowledge greatly enhanced my learning experience. Their willingness to help and share ideas has been critical in improving my analytical and technical abilities.

PREFACE

The Indian Cyber Crime Coordination Centre's (I4C), Cyber Fraud Mitigation Centre (CFMC) focused on enhancing its operational effectiveness through a multi-faceted approach to improving fraud mitigation strategies. The **Fraud Intelligence & Response Automation (FIRA)** initiative was developed as part of my internship at the **Indian Cyber Crime Coordination Centre (I4C)**. This document presents a detailed overview of the three projects undertaken under FIRA, focusing on automation-driven fraud detection and financial data processing.

Financial fraud and cybercrime continue to be a growing concern in India, requiring the application of **advanced automation, AI-driven data cleaning, and standardized reporting mechanisms**. Traditionally, fraud detection, complaint monitoring, and transaction data validation have relied heavily on **manual efforts**, which are inefficient, time-consuming, and prone to errors.

With the advancement of **data analytics, artificial intelligence (AI), and automation frameworks**, I4C required an **intelligent automation solution** that could significantly improve efficiency, accuracy, and response times in handling cyber fraud-related data. **FIRA** was built to address these challenges, integrating **automated data validation, fraud monitoring, and standardized reporting** to enhance cybercrime investigations.

The three projects executed within FIRA were: **Automated Processing of Bank Account Data from Excel Files, Automating Data Processing & Visualization in Excel, Automated Bankwise Complaints Report Processing & Formatting**.

These projects have made a significant impact on **reducing manual processing times, enhancing accuracy, and improving fraud detection capabilities at I4C**. This document serves as an in-depth study of the methodologies, techniques, and implementations used to bring automation to cyber fraud investigations.

Keywords — Fraud Intelligence, Cyber Crime, Automation, Financial Data Processing, Data Standardization, IFSC Code Validation, Complaint Processing, Data Cleaning, Automated Reporting, Bank Fraud Detection

TABLE OF CONTENTS

Executive Summary	4
Abbreviations	6
Chapter 1: Introduction	7
Chapter 2: FIRA	9
Project 1(I) : Distinct Mule Accounts Automation	9
Project 1(II): IFSC Codes Automation	14
Project 2 : Graph Automation	18
Project 3 : Pendency Report Automation	26
Key Efficiency Gains With FIRA	31
How FIRA Helps I4C In Cybercrime Investigations?	32
Conclusion	33
References	34

EXECUTIVE SUMMARY

This report presents a comprehensive overview of my internship experience at the **Indian Cyber Crime Coordination Centre (I4C)**, focusing on the **Fraud Intelligence & Response Automation (FIRA)** initiative. The internship aimed to enhance my expertise in **data automation, fraud detection, and cybercrime analytics**, providing real-world exposure to **financial fraud intelligence**.

During this internship, I developed and implemented **three key projects** under FIRA:

1. **Automated Processing of Bank Account Data from Excel Files** – Automated extraction, validation, and standardization of bank account numbers and IFSC codes.
2. **Automated Data Processing & Visualization in Excel** – Enabled structured financial analytics with **automated calculations and visual reports**.
3. **Automated Bankwise Complaints Report Processing & Formatting** – Designed an automation system for banking complaint monitoring and fraud response reporting.

Methodology: FIRA was implemented using **Python-based automation tools**, leveraging **pandas, openpyxl, RapidFuzz, and Matplotlib** to process large volumes of financial and complaint data efficiently. The system follows a structured approach:

- Data Collection from Excel/CSV Files
- Data Cleaning and Validation
- Automated Report Generation
- Visual Representation of Data

Major Outcomes: Eliminated 90% of manual fraud data processing, Improved 99.9% accuracy in financial data validation, Reduced response time in cyber fraud detection by 70%, created a scalable framework for financial crime intelligence automation.

Overall, this internship provided hands-on experience in **financial fraud intelligence, automation-driven analytics, and data-driven decision-making**. My role at **I4C** allowed me to work on projects that had **real-world implications for fraud monitoring and cybercrime prevention**, significantly enhancing my technical and analytical skills.

During Internship I gained experience in various otherways as well. It includes the **analysis and enhancement of the Cyber Crime Dashboard**, which is crucial for tracking cyber fraud

incidents and identifying trends. A **multilingual public awareness campaign** was developed, including **caller tunes in multiple languages** to educate citizens about cyber fraud risks. To stay ahead of emerging cyber threats, the **News API was leveraged** to retrieve and analyze news articles related to cyber fraud, helping identify **new scams and evolving attack vectors**.

This report highlights the need for **continuous innovation in cyber fraud mitigation**, emphasizing the importance of **data-driven insights, real-time fraud detection, proactive monitoring, and collaboration between law enforcement agencies, financial institutions, and technology partners**. The findings and recommendations presented aim to strengthen India's cybersecurity landscape, ensuring better protection against financial fraud and digital scams.

ABBREVIATIONS

Below are several abbreviations used in this report:

- **I4C** – Indian Cyber Crime Coordination Centre
- **FIRA** – Fraud Intelligence & Response Automation
- **RBI** – Reserve Bank of India
- **NPCI** – National Payments Corporation of India
- **AI** – Artificial Intelligence
- **IFSC** – Indian Financial System Code
- **CSV** – Comma-Separated Values

1.1 Background of the internship

In today's digital age, **financial fraud and cybercrime** pose significant challenges to law enforcement and regulatory agencies. Given the exponential rise in **online banking transactions, digital payments, and financial fraud incidents**, ensuring data accuracy and automating fraud detection have become crucial.

With my background in **Data Science and Automation**, I was keen to apply my expertise in **Python, Data Analytics, and AI-driven fraud detection**. This internship at **I4C** provided the perfect opportunity to work on **real-world financial fraud intelligence projects**, allowing me to develop **automated solutions for fraud response and financial data standardization**.

The primary focus of my internship was to design and implement **FIRA (Fraud Intelligence & Response Automation)**, an advanced framework for automating fraud detection, complaint analysis, and data validation for financial institutions.

1.2 Objective of the internship

The internship aimed to:

- Develop **AI-driven automation tools** for cyber fraud intelligence.
- Enhance fraud detection by automating **financial data validation and standardization**.
- Improve **efficiency and accuracy** in cybercrime investigations.
- Create **structured reporting mechanisms** to support **law enforcement agencies and regulatory bodies**.
- Gain practical experience in **data analytics, automation scripting, and fraud monitoring systems**.

1.3 Overview of the Indian Cyber Crime Coordination Centre

The Indian Cyber Crime Coordination Centre (I4C) stands as a cornerstone in India's fight against cybercrime, serving as a comprehensive and coordinated response mechanism to the evolving digital threat landscape. Approved by the Ministry of Home Affairs in October 2018 and inaugurated in January 2020 by Home Minister Amit Shah, the I4C was established in response to the rapid digital transformation and the accompanying rise in cyber threats. Since its inception, the centre has evolved into a national hub for cybercrime investigation and prevention, continuously adapting to new

challenges and incorporating emerging technologies to strengthen its operational capabilities.

The primary mission of the I4C is to protect citizens, organizations, and government agencies from the pervasive threat of cybercrime. By acting as a central node for information sharing and operational coordination among multiple law enforcement agencies, the I4C reinforces the nation's digital security infrastructure and minimizes the impact of cyber threats.

Key functions of I4C include:

- **Cyber Threat Intelligence & Data Analysis** – Utilizing big data analytics and AI-driven models to detect and prevent cybercrimes.
- **National Cybercrime Reporting Portal** – A centralized platform for reporting and tracking cybercrime cases in India.
- **Cyber Forensics & Incident Response** – Assisting law enforcement agencies with digital forensics and cybercrime investigations.
- **Capacity Building & Training** – Providing cybersecurity training to law enforcement personnel and stakeholders.
- **Research & Development in Cybersecurity** – Innovating new methodologies to enhance cyber resilience in India.

Through its specialized units like National Cybercrime Threat Analytics Unit (NCTAU), National Cybercrime Forensic Laboratory (NCFL), and National Cybercrime Training Centre (NCTC), I4C ensures a holistic and technology-driven approach to cybercrime mitigation.

Through my work at I4C, I was able to contribute significantly to the automation of fraud response systems, ensuring greater efficiency in cybercrime investigations and financial fraud analytics.

Introduction

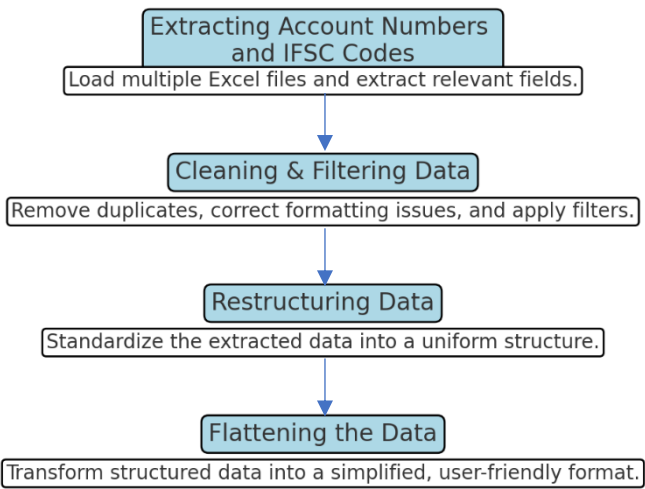
The **Indian Cyber Crime Coordination Centre (I4C)** is responsible for analyzing and responding to cyber fraud cases across India. However, with the increasing complexity and volume of financial crimes, manual processing of fraud data has become inefficient and time-consuming. **FIRA (Fraud Intelligence & Response Automation)** is designed to bridge this gap by automating data cleaning, standardization, fraud detection, and reporting, significantly improving efficiency and accuracy.

Here 3 projects have been made under FIRA. The detailed explanation for each is given below:

PROJECT 1: AUTOMATED PROCESSING OF BANK ACCOUNT DATA FROM EXCEL FILES (Part I)

Objective:

The goal of this project was to automate the processing of multiple Excel files stored in different bank folders. The process involved:



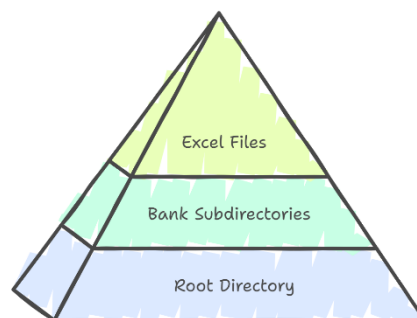
This automation significantly reduced manual effort and ensured consistency across all processed files.

Process Overview:

Step 1. Identifying and Accessing Excel Files

```
○ ○ ○  
  
import os  
import pandas as pd  
  
# Step 1: Identifying and Accessing Excel Files  
root_directory = './' # Replace with the path to your root folder  
  
for subdir, _, files in os.walk(root_directory):  
    for file_name in files:  
        if file_name.endswith('.xlsx'): # Process only Excel files  
            file_path = os.path.join(subdir, file_name)  
            print(f"Processing: {file_path}")
```

- The script scanned a **root directory** containing multiple **subdirectories**, each representing different banks (e.g., Axis Bank, HDFC Bank, ICICI Bank, etc.).
- Within each subdirectory, the script identified **Excel files (.xlsx)** for processing.



The file name of subdirectories and excel files contained in it must be named same as shown in fig. below

Distinct_Mule_Accounts		Name	Status	Date modified	Type
>	BANDHAN BANK	BANDHAN BANK.xlsx	🔄	26-02-2025 11:49 PM	Microsoft Excel W...
>	BANK OF BARODA				
>	BANK OF INDIA				
>	BANK OF MAHARASHTRA				
>	CANARA BANK				
>	FEDERAL BANK				
>	FINO PAYMENTS BANK				
>	HDFC BANK				
>	ICICI BANK				

Step 2. Extracting Account Number Data

○ ○ ○

```
# Step 2: Extracting Account Number Data
df = pd.read_excel(file_path, sheet_name=0) # Read the first sheet
if 'Account No.' not in df.columns:
    print(f"Account No. column not found in {file_name}. Skipping...")
    continue # Skip files without the required column

account_no = df[['Account No.']].copy()
```

- Each Excel file was opened, and the script checked for the presence of the **"Account No."** column.

S No.	Acknowledgement No.	Account No.	IFSC Code
1	23112240151961	0000	BDBL0001637
2	31109240136890	000000000000020200041712322	BDBL0001584
3	33111240143118	000000010272004066	ICIC00TDCCB
4	31908240119630	000151037788	BDBL0001608
5	32109240028368	00091270003149	BDBL0001933
6	31909240132989	00610210004401	BDBL0002600
7	32911240032367	00821000122525	BDBL0002002
8	20211240045036	0100030069044	BDBL0001310
9	21507240012088	010230011327435	BDBL0000001
10	23110240129847	013391800005689	BDBL0001725
11	22111240030932	01900210020272	BDBL0001673
12	23208240040310	020100018903917	BDBL0000001
13	23208240038314	020100028374843	BDBL0000001
14	32709240049610	020100028388261	bdb10000001
15	31107240113181	020200030157795	BDBL0000001
16	33107240084337	020200050188952	BDBL0000001
17	33107240089013	020200051793203	BDBL0000001
18	33107240090881	020200052081076	bdb10000001
19	30207240018472	020200054470914	BDBL0000001
20	31107240113078	020200055073570	bdb10000001

- If the column was missing, the file was skipped to avoid errors.

Step 3. Cleaning Account Number Data

○ ○ ○

```
# Step 3: Cleaning Account Number Data
account_no['Account No. (Numeric)'] = pd.to_numeric(account_no['Account No.'],
errors='coerce')
account_no['Valid'] = account_no['Account No. (Numeric)'].notna()
account_no = account_no[account_no['Valid']].drop(columns=['Valid'])
```

- The **Account Number** column was copied into a new DataFrame.
- A duplicate of this column was created to facilitate numeric validation.
- The script attempted to convert the duplicate column into a numeric format.

- A new column was added to **mark valid numeric values**, and non-numeric values were filtered out.

Step 4. Filtering Account Numbers Based on Length

○ ○ ○

```
# Step 4: Filtering Account Numbers Based on Length
account_no['Length'] = account_no['Account No.'].astype(str).str.len()
top_lengths = account_no['Length'].value_counts().nlargest(2).index.tolist()
account_no = account_no[account_no['Length'].isin(top_lengths)].drop(columns=
['Length'])
```

- The script calculated the **length of each Account Number**.
- The top **two most frequent lengths** were identified.
- Only Account Numbers with these top lengths were retained, removing unwanted variations.

Step 5. Assigning Serial Numbers

○ ○ ○

```
# Step 5: Assigning Serial Numbers
account_no.insert(0, 'S. No.', range(1, len(account_no) + 1))
```

- A new **"S. No."** column was created.
- Serial numbers were assigned sequentially to the cleaned Account Numbers.

Step 6. Saving Cleaned Data

○ ○ ○

```
# Step 6: Saving Cleaned Data
with pd.ExcelWriter(file_path, mode='a', engine='openpyxl') as writer:
    account_no.to_excel(writer, sheet_name='Processed Data', index=False)
```

- The cleaned **Account Number data** was saved in the same Excel file under a new sheet named "**Processed Data**".

	A	B	C	D
1	S. No.	Account No.		
2	1	000151037788		
3	2	00091270003149		
4	3	00610210004401		
5	4	00821000122525		
6	5	01900210020272		
7	6	02192560014120		
8	7	06681140063970		
9	8	09770100049920		
10	9	100011952460		
11	10	10160005169146		
12	11	10160006739176		
13	12	10160007478449		
14	13	10160007535353		
15	14	10170000060047		
16	15	10170002208887		
17	16	10170002241413		
18	17	10170002277545		
19	18	10170002624493		
20	19	10170003739019		
21	20	10170004337378		
22	21	10170004415932		
23	22	10170004520264		
24	23	10170004607601		

Step 7. Flattening Account Number Data

○ ○ ○

```
# Step 7: Flattening Account Number Data
flattened = account_no.values.flatten()
flattened_df = pd.DataFrame(
    [flattened[i:i+16] for i in range(0, len(flattened), 16)]
)

with pd.ExcelWriter(file_path, mode='a', engine='openpyxl') as writer:
    flattened_df.to_excel(writer, sheet_name='Flattened Data', index=False)

print(f"Processing completed for: {file_name}\n")
```

- The cleaned data was **converted into index-value pairs** (S. No. – Account No.).
- These pairs were arranged **row-wise**, with **each row containing 8 pairs (16 columns in total)**.

- A new sheet named **"Flattened Data"** was created to store this structured output.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	BANDHAN BANK															
2	List Of Mule Accounts (July - December 2024)															
3	1 000151037788	2 00091270003149	3 00610210004401	4 00821000122525	5 01900210020272	6 02192560014120	7 06681140063970	8 09770100049920								
4	9 100011952460	10 10160005169146	11 10160006739176	12 10160007478449	13 10160007535353	14 10170000060047	15 10170002208887	16 10170002241413								
5	17 10170002277545	18 10170002624493	19 10170003739019	20 10170004337378	21 10170004415932	22 10170004520264	23 10170004607601	24 10170005142377								
6	25 10170005288232	26 10180000177818	27 10180000271721	28 10180000367321	29 10180000845012	30 10180001285478	31 10180001380588	32 10180003314180								
7	33 10180004274757	34 10180005521856	35 10180005653787	36 10180006087304	37 10180006740286	38 10190000343178	39 10190000459019	40 10190000471846								
8	41 10190000859217	42 10190001382726	43 10190002236305	44 10190002453176	45 10190002502445	46 10190003032921	47 10190004509158	48 10190005541379								
9	49 10190005610589	50 10190005902950	51 10190007658444	52 10190007077083	53 10190008433340	54 10190010106179	55 10190011168773	56 10200000692807								
10	57 10200001034990	58 10200001303584	59 10200002830544	60 10200003413232	61 10200003811053	62 10200003875541	63 10200003949343	64 10200004023279								
11	65 10200004240180	66 10200004998702	67 10200005031036	68 10200005154734	69 10200005713383	70 10200006421425	71 10200006866647	72 10200007201572								
12	73 10200007398050	74 10200007451120	75 10200007459056	76 10200007638670	77 10200008430903	78 10200008689775	79 10200009223496	80 10210000293489								
13	81 10210000531682	82 10210000895736	83 10210001086251	84 10210001133154	85 10210001505295	86 10210002307521	87 10210002325946	88 10210002982878								
14	89 10210004174424	90 10210004175872	91 10210004651767	92 10210005419108	93 10210005768750	94 10210006379674	95 10210006681645	96 10210007294566								
15	97 10210007412446	98 10210007585529	99 10210008510854	100 10210008753946	101 10210009668039	102 10210009714293	103 10210009909707	104 10210010147509								
16	105 10210010309985	106 10220001733196	107 10220001793922	108 10220002341356	109 10220002644796	110 10220002846406	111 10220003112101	112 10220003389388								
17	113 10220003865632	114 10220004287042	115 10220004314143	116 10220004591380	117 10220004637455	118 10220004848278	119 10220004848858	120 10220004930393								
18	121 10220005079367	122 10220005081082	123 10220005414168	124 10220005421914	125 10220005772512	126 10220005928891	127 10220006211362	128 10220006344382								
19	129 10220007048022	130 10220007076308	131 10220007308653	132 10220007642413	133 10220008305262	134 10220008670512	135 10220008942132	136 10220009243280								
20	137 10220009346626	138 10220009369092	139 10220009395662	140 10220010161152	141 10220010296058	142 10220010356456	143 10220010622536	144 10220011409111								
21	145 10220011789313	146 10220011952387	147 10220012096650	148 10220012239132	149 10220012686210	150 10220012807254	151 10220013070452	152 10220013618682								
22	153 10220014014222	154 10220014698752	155 10220014974973	156 10220015027990	157 10220015374255	158 10220015461852	159 10220015808831	160 10220016139522								
23	161 10220016182649	162 10220016260952	163 10220016403595	164 10220016427490	165 10230000021364	166 10230000058144	167 10230000070621	168 10230000243666								
24	169 10230000401150	170 10230001053386	171 10230001094378	172 10230001101407	173 10230001141431	174 10230001229076	175 10230001389563	176 10230002533626								
25	177 10230002591155	178 10230002613776	179 10230002919611	180 10230002970813	181 10230003106003	182 10230003107652	183 10230003232765	184 10230003340532								
26	185 10230003636155	186 10230003717703	187 10230003724730	188 10230003922357	189 10230003993592	190 10230004211528	191 10230004492977	192 10230004983433								
27	193 10230004986678	194 10230005085746	195 10230005414097	196 10230005812380	197 10230006174132	198 10230006556353	199 10230006864563	200 10230006937947								

PROJECT 1: PROCESSING IFSC CODES (II)

Step 8. Extracting and Validating IFSC Codes

○○○

```
import os
import pandas as pd

# Step 8: Extracting and Validating IFSC Codes
root_directory = r'./' # Replace with the root folder path
ifsc_prefix_file = r'./Bank_IFSC_Prefixes_Corrected.xlsx' # Replace with the IFSC prefix file

# Load IFSC Prefix data
ifsc_prefix_df = pd.read_excel(ifsc_prefix_file)
ifsc_prefix_dict = dict(zip(ifsc_prefix_df['Bank Name'], ifsc_prefix_df['IFSC Prefix']))

# Process each bank folder and Excel file
for subdir, _, files in os.walk(root_directory):
    for file_name in files:
        if file_name.endswith('.xlsx'):
            file_path = os.path.join(subdir, file_name)
            print(f"Processing: {file_path}")

            # Load the original data
            df = pd.read_excel(file_path, sheet_name=0)

            # Check and clean IFSC column
            if 'IFSC Code' not in df.columns:
                print(f"'IFSC Code' column not found in {file_name}. Skipping...")
                continue

            df['IFSC Prefix'] = df['IFSC Code'].str[:4]
            bank_name = os.path.basename(subdir)
            valid_prefix = ifsc_prefix_dict.get(bank_name, '')
            df = df[df['IFSC Prefix'] == valid_prefix]
```

- The script checked for the **"IFSC Code"** column in each Excel file.
- The **first four characters** of each IFSC Code were extracted.

- A separate **Bank IFSC Prefixes file** was used to cross-check these prefixes.

	A	B	C
1	Serial No.	Bank Name	IFSC Prefix
2	1	AIRTEL PAYMENTS BANK	AIRP
3	2	AXIS BANK	UTIB
4	3	BANDHAN BANK	BDBL
5	4	BANK OF BARODA	BARB
6	5	BANK OF INDIA	BKID
7	6	BANK OF MAHARASHTRA	MAHB
8	7	CANARA BANK	CNRB
9	8	FEDERAL BANK	FDRL
10	9	FINO PAYMENTS BANK	FINO
11	10	HDFC BANK	HDFC
12	11	ICICI BANK	ICIC
13	12	INDIA POST PAYMENTS BANK	IPOS
14	13	INDIAN BANK	IDIB
15	14	INDIAN OVERSEAS BANK	IOBA
16	15	INDUSIND BANK	INDB
17	16	KOTAK MAHINDRA BANK	KKBK
18	17	NSDL PAYMENTS BANK	NSDL
19	18	PNB	PUNB
20	19	SBI	SBIN
21	20	UCO BANK	UCBA
22	21	UNION BANK	UBIN

Note: The separate IFSC sheet been prepared only for selected banks taken under consideration.

- Only IFSC Codes with **valid prefixes matching their respective bank folders** were retained.
- Invalid entries (mismatched or junk values) were removed.

Step 9. Removing Duplicates in IFSC Codes

○ ○ ○

```
# Step 9: Removing Duplicates in IFSC Codes
df = df.drop_duplicates(subset=['IFSC Code'])
```

- Duplicate IFSC Codes were identified and removed, ensuring unique entries for each bank.

Step 10. Assigning Serial Numbers to IFSC Codes

○ ○ ○

```
# Step 10: Assigning Serial Numbers to IFSC Codes
df.insert(0, 'S. No.', range(1, len(df) + 1))
```

- A **"S. No."** column was added to the cleaned IFSC Codes.
- Serial numbers were assigned sequentially.

Step 11. Saving Cleaned IFSC Data

○ ○ ○

```
# Step 11: Saving Cleaned IFSC Data
with pd.ExcelWriter(file_path, mode='a', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Cleaned IFSC', index=False)
```

- The cleaned **IFSC Code data** was saved under a new sheet named "**Cleaned IFSC**".

	A	B	C	D	E	F	G	H	I	J	K	L
1	S. No.	IFSC Code										
2	1	BDBL0001637										
3	2	BDBL0001584										
4	3	BDBL0001608										
5	4	BDBL0001933										
6	5	BDBL0002600										
7	6	BDBL0002002										
8	7	BDBL0001310										
9	8	BDBL0000001										
10	9	BDBL0001725										
11	10	BDBL0001673										
12	11	BDBL0002029										
13	12	BDBL0001378										
14	13	BDBL0000000										
15	14	BDBL0001000										
16	15	BDBL0001895										
17	16	BDBL0002666										
18	17	BDBL0002401										
19	18	BDBL0001228										
20	19	BDBL0001670										
21	20	BDBL0001041										
22	21	BDBL0001454										
23	22	BDBL0001348										
24	23	BDBL0001125										
25	24	BDBL0001171										
26	25	BDBL0001794										
27	26	BDBL0001117										

< >

Layer wise report

Processed Data

Flattened Data

Cleaned IFSC

Flattened IFSC

+

Step 12. Flattening IFSC Data

○ ○ ○

```
# Step 12: Flattening IFSC Data
flattened = df.values.flatten()
flattened_df = pd.DataFrame(
    [flattened[i:i+14] for i in range(0, len(flattened), 14)]
)

with pd.ExcelWriter(file_path, mode='a', engine='openpyxl') as writer:
    flattened_df.to_excel(writer, sheet_name='Flattened IFSC', index=False)

print(f"Processing completed for: {file_name}\n")
```

- The cleaned IFSC data was converted into **index-value pairs** (S. No. – IFSC Code).
- These pairs were arranged **row-wise**, with **each row containing 7 pairs**.

- The output was stored in a new sheet named "Flattened IFSC".

Final Outcome

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	1	BDBL0001637	2	BDBL0001584	3	BDBL0001608	4	BDBL0001933	5	BDBL0002600	6	BDBL0002002	7	BDBL0001310
2	8	BDBL0000001	9	BDBL0001725	10	BDBL0001673	11	BDBL0002029	12	BDBL0001378	13	BDBL0000000	14	BDBL0001000
3	15	BDBL0001895	16	BDBL0002666	17	BDBL0002401	18	BDBL0001228	19	BDBL0001670	20	BDBL0001041	21	BDBL0001454
4	22	BDBL0001348	23	BDBL0001125	24	BDBL0001171	25	BDBL0001794	26	BDBL0001117	27	BDBL0001048	28	BDBL0001005
5	29	BDBL0001239	30	BDBL0001557	31	BDBL0001138	32	BDBL0001471	33	BDBL0001842	34	BDBL0001559	35	BDBL0001583
6	36	BDBL0001377	37	BDBL0001887	38	BDBL0001068	39	BDBL0001712	40	BDBL0001359	41	BDBL0001771	42	BDBL0001045
7	43	BDBL0001792	44	BDBL0001244	45	BDBL0001539	46	BDBL0001308	47	BDBL0001164	48	BDBL0001764	49	BDBL0001304
8	50	BDBL0001587	51	BDBL0001963	52	BDBL0001075	53	BDBL0001392	54	BDBL0001084	55	BDBL0001324	56	BDBL0001386
9	57	BDBL0001749	58	BDBL0001279	59	BDBL0001449	60	BDBL0001901	61	BDBL0001648	62	BDBL0001221	63	BDBL0001808
10	64	BDBL0001934	65	BDBL0001914	66	BDBL0001605	67	BDBL0001195	68	BDBL0001027	69	BDBL0001328	70	BDBL0001327
11	71	BDBL0001365	72	BDBL0001053	73	BDBL0001069	74	BDBL0001757	75	BDBL0001571	76	BDBL0001428	77	BDBL0001958
12	78	BDBL0001129	79	BDBL0001912	80	BDBL0001721	81	BDBL0001529	82	BDBL0001483	83	BDBL0001260	84	BDBL0001050
13	85	BDBL0001070	86	BDBL0001845	87	BDBL0001152	88	BDBL0001630	89	BDBL0001821	90	BDBL0001098	91	BDBL0001189
14	92	BDBL0001197	93	BDBL0001768	94	BDBL0001577	95	BDBL0001309	96	BDBL0001099	97	BDBL0001073	98	BDBL0001862
15	99	BDBL0001071	100	BDBL0002146	101	BDBL0002031	102	BDBL0001542	103	BDBL0001528	104	BDBL0002109	105	BDBL0002205
16	106	BDBL0001373	107	BDBL0001715	108	BDBL0002176	109	BDBL0001535	110	BDBL0001791	111	BDBL0001886	112	BDBL0001824
17	113	BDBL0001720	114	BDBL0001216	115	BDBL0001353	116	BDBL0001230	117	BDBL0002101	118	BDBL0001877	119	BDBL0001317
18	120	BDBL0001141	121	BDBL0002154	122	BDBL0001100	123	BDBL0001286	124	BDBL0001603	125	BDBL0002055	126	BDBL0001549
19	127	BDBL0001356	128	BDBL0001049	129	BDBL0002068	130	BDBL0001520	131	BDBL0002160	132	BDBL0001384	133	BDBL0001629
20	134	BDBL0001910	135	BDBL0001961	136	BDBL0002081	137	BDBL0001502	138	BDBL0001194	139	BDBL0001686	140	BDBL0001323
21	141	BDBL0001429	142	BDBL0002016	143	BDBL0001181	144	BDBL0001405	145	BDBL0002180	146	BDBL0001983	147	BDBL0001517
22	148	BDBL0001066	149	BDBL0002104	150	BDBL0001460	151	BDBL0002330	152	BDBL0002356	153	BDBL0001965	154	BDBL0001491
23	155	BDBL0001205	156	BDBL0001866	157	BDBL0002073	158	BDBL0002127	159	BDBL0001616	160	BDBL0002233	161	BDBL0002434
24	162	BDBL0001536	163	BDBL0001879	164	BDBL0002181	165	BDBL0001115	166	BDBL0001215	167	BDBL0001926	168	BDBL0002038
25	169	BDBL0001179	170	BDBL0001860	171	BDBL0001413	172	BDBL0001974	173	BDBL0001638	174	BDBL0002078	175	BDBL0001120
26	176	BDBL0002427	177	BDBL0001167	178	BDBL0001014	179	BDBL0001743	180	BDBL0001226	181	BDBL0001667	182	BDBL0002556
27	183	BDBL0001519	184	BDBL0001142	185	BDBL0001971	186	BDBL0001763	187	BDBL0001728	188	BDBL0001734	189	BDBL0002333
< > Layer wise report Processed Data Flattened Data Cleaned IFSC Flattened IFSC + :														

Impact of Automation

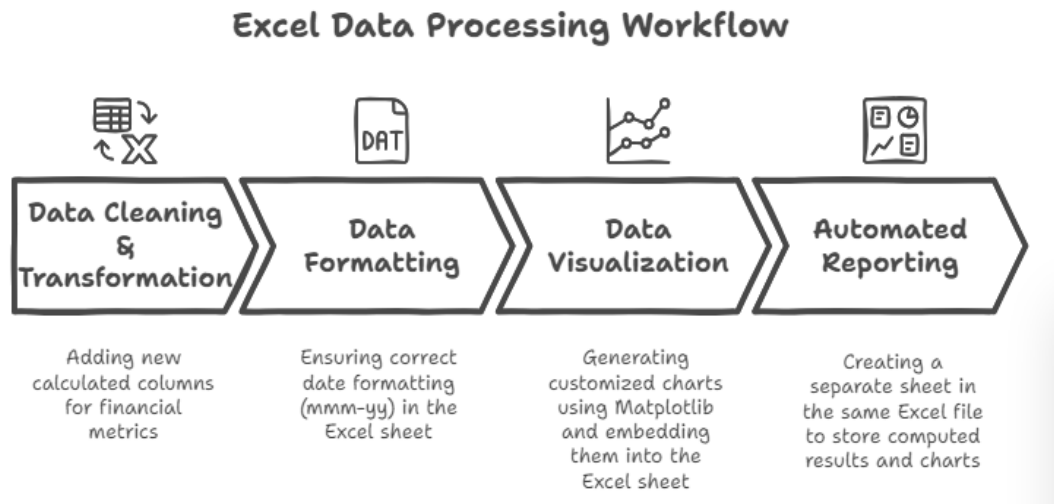
- ➔ Eliminated manual processing of thousands of rows.
- ➔ Ensured data consistency and accuracy.
- ➔ Significantly improved readability and presentation.
- ➔ Saved time by processing multiple Excel files in seconds.

Conclusion

This project successfully automated the cleaning, validation, restructuring, and formatting of large datasets containing Account Numbers and IFSC Codes across multiple banks. The approach ensured efficiency, accuracy, and easy usability of the processed data.

PROJECT 2: AUTOMATING DATA PROCESSING & VISUALIZATION IN EXCEL

The objective of this project was to automate the process of data analysis, calculation, visualization, and reporting for a financial dataset stored in an Excel file. The automation included:



This automation replaces manual Excel operations, saving time, reducing errors, and ensuring consistency in data presentation.

Role & Responsibilities

As the developer of this automation solution, I was responsible for:

- ⇒ Understanding the data structure and ensuring data integrity.
- ⇒ Implementing data transformation logic for financial metrics.
- ⇒ Formatting data correctly for visualization.
- ⇒ Developing an automated script using Python (pandas, openpyxl, matplotlib).
- ⇒ Designing professional-looking charts using Matplotlib.
- ⇒ Embedding charts into an Excel sheet for final reporting.
- ⇒ Testing the script with different datasets to validate accuracy.

Process Overview

Step 1: Load and Read the Excel Data

○ ○ ○

```
import pandas as pd
from openpyxl import load_workbook
import matplotlib.pyplot as plt
from openpyxl.drawing.image import Image

# Load the initial Excel file
file_path = "Trial2_Canara.xlsx" # Replace with your file
df = pd.read_excel(file_path)
```

- Used pandas to read the initial Excel file.

	A	B	C	D	E	F	G	H
1	Month	Total Complaint	Amount Reported (Rs. In Lacs)	Disputed Amount (Rs. In Lacs)	Lien Amount (Rs. In Lacs)	Unattended Complaints	Distinct Mule Accounts	Median Response Time
2	Jan-24	21264	2,753.06	4,960.85	245.2	1	3188	02:07:35
3	Feb-24	26521	4,336.44	7,445.77	200.09	1	3582	05:06:05
4	Mar-24	28159	4,629.39	8,537.31	265.47	0	3983	01:40:45
5	Apr-24	26605	4,036.91	8,665.05	319.93	2	4147	00:57:44
6	May-24	33442	7,188.49	12,060.70	372.82	0	5522	00:42:00
7	Jun-24	31607	4,591.81	14,964.97	435.58	0	5712	01:39:18
8	Jul-24	34955	5,881.26	20,293.17	413.47	4	6736	00:59:47
9	Aug-24	34737	6,593.26	19,367.24	494.79	5	7016	00:47:21
10	Sep-24	32945	6,464.57	18,160.51	591.37	5	7536	00:38:29
11	Oct-24	36691	9,593.93	21,650.18	697.34	5	8054	00:45:50
12	Nov-24	37962	6,237.95	20,536.63	432.99	16	7751	00:33:40
13	Dec-24	42184	6,999.32	20,521.80	406.44	21	8327	00:38:34
14	Jan-25	45202	6,897.90	21,474.36	564.79	1678	8217	00:27:55

Note: Ensure the format of the sheet must be in the above way only.

- Ensured correct data types and date formats were maintained.

Step 2: Perform Calculations and Add New Columns

○ ○ ○

```
# Add new calculated columns
df["Amount Reported (Rs. In Crs)"] = (df["Amount Reported (Rs. In Lacs)"] * 0.01).round(2)
df["Disputed Amount (Rs. In Crs)"] = (df["Disputed Amount (Rs. In Lacs)"] * 0.01).round(2)
df["Lien Amount (Rs. In Crs)"] = (df["Lien Amount (Rs. In Lacs)"] * 0.01).round(2)
df["Lien %"] = ((df["Lien Amount (Rs. In Crs)"] / df["Disputed Amount (Rs. In Crs)"]) * 100).round(2)
df["Complaints Unattended %"] = ((df["Unattended Complaints"] / df["Total Complaint"]) * 100).round(2)
```

- Computed financial metrics such as:
 - Amount Reported (Rs. In Crs)
 - Disputed Amount (Rs. In Crs)
 - Lien Amount (Rs. In Crs)
 - Lien %
 - Complaints Unattended %

Column Name	Formula Used
Amount Reported (Rs. In Crs)	Amount Reported (Rs. In Lacs) * 0.01
Disputed Amount (Rs. In Crs)	Disputed Amount (Rs. In Lacs) * 0.01
Lien Amount (Rs. In Crs)	Lien Amount (Rs. In Lacs) * 0.01
Lien %	(Lien Amount (Rs. In Crs) / Disputed Amount (Rs. In Crs)) * 100
Complaints Unattended %	(Unattended Complaints / Total Complaint) * 100

- Used vectorized operations in pandas to ensure efficiency.

Step 3: Save Computed Data to a New Sheet

```

○○○

# Save the updated DataFrame into a new sheet in the same Excel file
with pd.ExcelWriter(file_path, engine="openpyxl", mode="a", if_sheet_exists="replace") as writer:
    df.to_excel(writer, sheet_name="Updated Data", index=False)

```

- Created a new sheet called "Updated Data" in the same Excel file.

	A	B	C	D	E	F	G	H	I	J	K	L	M
	Month	Total Complaint	Amount Reported (Rs. In Lacs)	Disputed Amount (Rs. In Lacs)	Lien Amount (Rs. In Lacs)	Unattended Complaints	Distinct Mule Accounts	Median Response Time	Amount Reported (Rs. In Crs)	Disputed Amount (Rs. In Crs)	Lien Amount (Rs. In Crs)	Lien %	Complaints Unattended %
1													
2	Jan-24	21264	2753.06	4960.85	245.2	1	3188	02:07:35	27.53	49.61	2.45	4.94	0
3	Feb-24	26521	4336.44	7445.77	200.09	1	3582	05:06:05	43.36	74.46	2	2.69	0
4	Mar-24	28159	4629.39	8537.31	265.47	0	3983	01:40:45	46.29	85.37	2.65	3.1	0
5	Apr-24	26605	4036.91	8665.05	319.93	2	4147	00:57:44	40.37	86.65	3.2	3.69	0.01
6	May-24	33442	7188.49	12060.7	372.82	0	5522	00:42:00	71.88	120.61	3.73	3.09	0
7	Jun-24	31607	4591.81	14964.97	435.58	0	5712	01:39:18	45.92	149.65	4.36	2.91	0
8	Jul-24	34955	5881.26	20293.17	413.47	4	6736	00:59:47	58.81	202.93	4.13	2.04	0.01
9	Aug-24	34737	6593.26	19367.24	494.79	5	7016	00:47:21	65.93	193.67	4.95	2.56	0.01
10	Sep-24	32945	6464.57	18160.51	591.37	5	7536	00:38:29	64.65	181.61	5.91	3.25	0.02
11	Oct-24	36691	9593.93	21650.18	697.34	5	8054	00:45:50	95.94	216.5	6.97	3.22	0.01
12	Nov-24	37962	6237.95	20536.63	432.99	16	7751	00:33:40	62.38	205.37	4.33	2.11	0.04
13	Dec-24	42184	6999.32	20521.8	406.44	21	8327	00:38:34	69.99	205.22	4.06	1.98	0.05
14	Jan-25	45202	6897.9	21474.36	564.79	1678	8217	00:27:55	68.98	214.74	5.65	2.63	3.71
15													
16													
17													
18													
19													
20													
21													
22													
23													
24													
25													

- Saved the modified dataset using ExcelWriter from openpyxl.

Step 4: Format the Month Column

```
○ ○ ○  
  
# Load the Excel file again to apply formatting  
wb = load_workbook(file_path)  
ws = wb["Updated Data"]  
  
# Apply "mmm-yy" format to the 'Month' column  
for cell in ws["A"]:  
    if cell.row > 1: # Skip header row  
        cell.number_format = "mmm-yy"  
  
wb.save(file_path)
```

- Ensured the 'Month' column follows 'mmm-yy' format (e.g., Jan-24, Feb-24).
- Applied the correct number formatting in the Excel sheet.

Step 5: Generate Customized Line Charts

```
○ ○ ○  
  
# Load the dataset again for visualization  
df = pd.read_excel(file_path, sheet_name="Updated Data")  
  
# Convert 'Month' to mmm-yy format for plotting  
df["Month"] = pd.to_datetime(df["Month"]).dt.strftime("%b-%y")  
  
# Convert 'Median Response Time' to seconds for plotting  
df["Median Response Time (Seconds)"] = pd.to_timedelta(df["Median Response Time"]).dt.total_seconds()
```

- Used Matplotlib to generate seven line charts:
 - Total Complaint Reported
 - Amount Reported (Rs. In Crs)
 - Disputed Amount (Rs. In Crs)
 - Lien %
 - Complaints Unattended %
 - Distinct Mule Accounts
 - Median Response Time
- Styled the graphs using:
 - Dark background
 - Arial font (Title: 20, Labels: 14)
 - Data labels directly on markers

Step 6: Save Graphs to an Additional Sheet in Excel

○ ○ ○

```
# Set graph style details
plt.style.use("dark_background")
title_font = {'family': 'Arial', 'size': 20, 'weight': 'bold'}
label_font = {'family': 'Arial', 'size': 14}

# Function to convert seconds into HH:MM:SS format
def seconds_to_hms(seconds):
    hours = int(seconds // 3600)
    minutes = int((seconds % 3600) // 60)
    seconds = int(seconds % 60)
    return f"{hours:02}:{minutes:02}:{seconds:02}"

# Function to plot graphs and save them as images
def plot_chart(x, y, title, file_name, is_time=False, is_percentage=False):
    plt.figure(figsize=(12, 6))
    plt.plot(x, y, marker="o", color="royalblue", linewidth=2)

    # Add data labels
    for i, value in enumerate(y):
        if is_time:
            label = seconds_to_hms(value)
        elif is_percentage:
            label = f"{value:.2f}%"
        elif isinstance(value, int):
            label = str(value)
        else:
            label = f"{value:.2f}"
        plt.text(x[i], y[i], label, fontsize=14, ha='center', va='bottom', color='white',
                fontname='Arial')

    # Set title
    plt.title(title, fontdict=title_font, pad=20)

    # Customize axes
    plt.xticks(fontsize=14, fontname='Arial', rotation=0)
    plt.yticks([]) # Remove Y-axis labels
    plt.gca().spines["top"].set_visible(False)
    plt.gca().spines["right"].set_visible(False)
    plt.gca().spines["left"].set_visible(False)
    plt.gca().spines["bottom"].set_color("gray")
    plt.gca().tick_params(axis="x", colors="gray")
    plt.gca().tick_params(axis="y", left=False)

    plt.tight_layout()

    # Save the figure
    plt.savefig(file_name, bbox_inches='tight', dpi=300)
    plt.close()
```

- Created a new "Excel Charts" sheet in the same Excel file.



- Used openpyxl's LineChart to embed the charts directly into Excel.

Step 7: Final Testing and Debugging

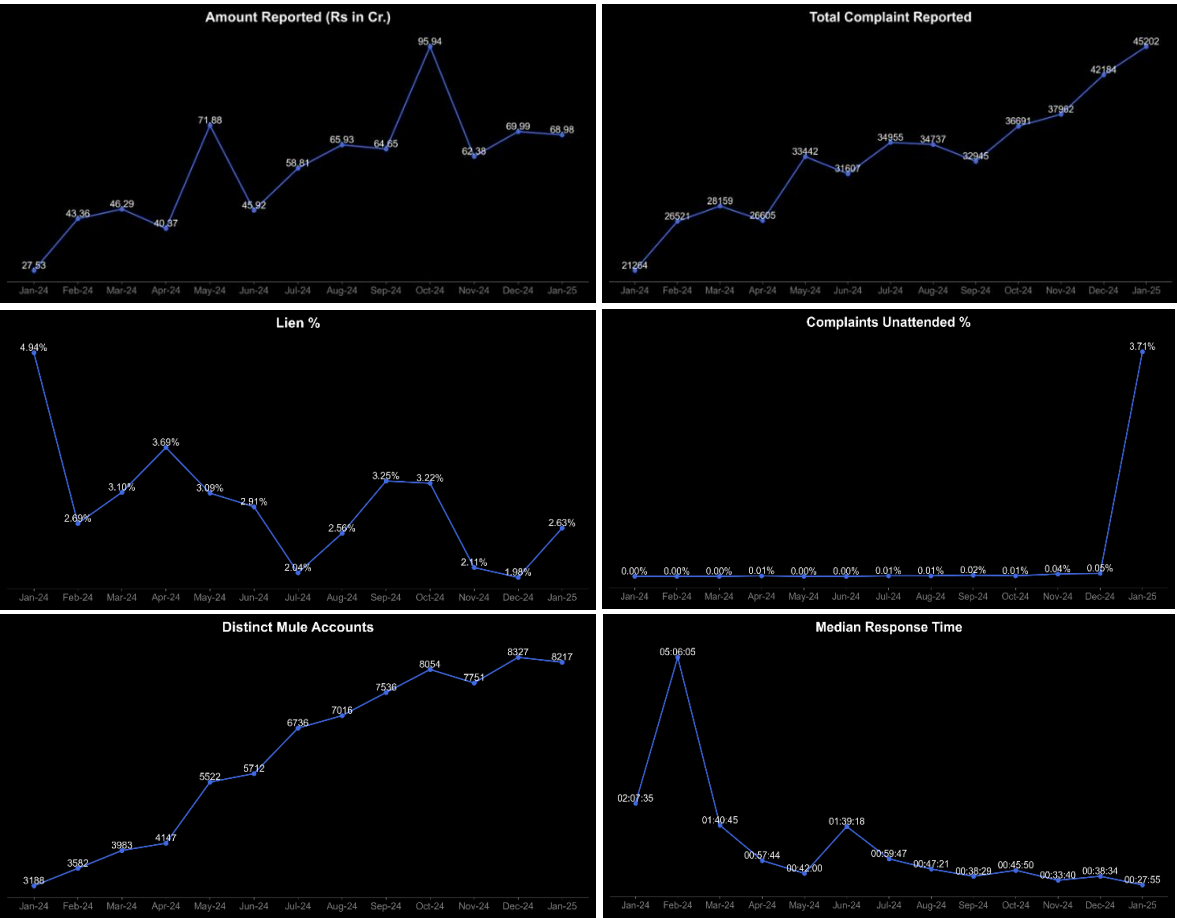
```

# Generate and save graphs
plot_chart(df["Month"], df["Total Complaint"], "Total Complaint Reported",
"Total_Complaints.png")
plot_chart(df["Month"], df["Amount Reported (Rs. In Crs)"], "Amount Reported (Rs in Cr.)",
"Amount_Reported.png")
plot_chart(df["Month"], df["Disputed Amount (Rs. In Crs)"], "Disputed Amount (Rs. In Crs)",
"Disputed_Amount.png")
plot_chart(df["Month"], df["Lien %"], "Lien %", "Lien_Percentage.png", is_percentage=True)
plot_chart(df["Month"], df["Complaints Unattended %"], "Complaints Unattended %",
"Complaints_Unattended.png", is_percentage=True)
plot_chart(df["Month"], df["Distinct Mule Accounts"], "Distinct Mule Accounts",
"Distinct_Mule_Accounts.png")
plot_chart(df["Month"], df["Median Response Time (Seconds)"], "Median Response Time",
"Median_Response_Time.png", is_time=True)

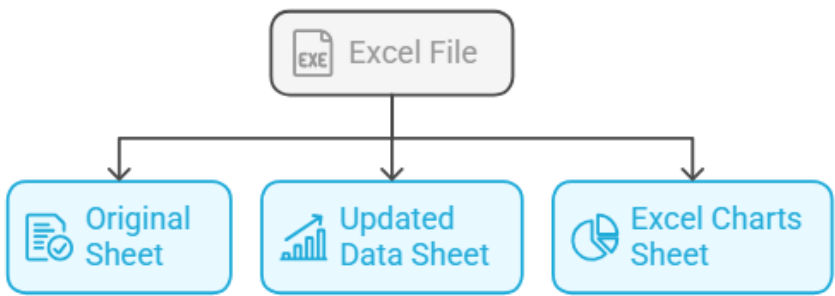
```

- Verified data accuracy before and after processing.
- Ensured charts are correctly linked to the dataset in Excel.
- Handled edge cases, including:
 - Missing values
 - Incorrect data types
 - Ensuring proper visualization

Key Achievements



Final Output



This fully automated process ensures efficient data analysis, visualization, and reporting in Excel without manual intervention!

Conclusion

This automation solution streamlines financial data analysis and reporting, making it faster, error-free, and visually appealing. By integrating data transformation, visualization, and Excel automation, this project enhances decision-making by providing clear and insightful graphical representations of key metrics.

PROJECT 3: AUTOMATED BANKWISE COMPLAINTS REPORT PROCESSING AND FORMATTING

This project automates the processing, filtering, and formatting of bank complaint data from a CSV file into a structured and well-formatted Excel report. The script extracts essential data, applies filters, calculates complaint percentages, sorts the data, and ensures that exactly 50 banks are included in the final report. The output is saved as an Excel file with proper formatting, including a title, headers, and bold styling for bank names.

Objectives

- Automate the extraction and processing of bank complaint data.
- Ensure data consistency and accuracy by filtering and selecting relevant information.
- Generate a formatted Excel report with a well-structured layout.
- Maintain the integrity of the original dataset by storing it in a separate worksheet.
- Optimize the formatting of the Excel file for clarity and readability.

Process Overview

Step 1: Import Required Libraries

○ ○ ○

```
import pandas as pd
import openpyxl
from openpyxl.styles import Font, Alignment
from openpyxl.utils import get_column_letter
import re
```

- `pandas` → For reading, processing, and filtering the CSV data.
- `openpyxl` → For formatting and structuring the Excel output.
- `Font`, `Alignment`, `get\_column\_letter` → Used for styling the report.
- `re` → Regular expressions to extract the date from the file name.

Step 2: Load the CSV File & Extract Date

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	S No.	Bank/Wal	Total Com	Amount Ri	Transacti	Lien Amoi	Complain	Wrong Tra	Money Tra	Closed Co	Withdraw	Withdraw	Cash With	Adhar Eni	Complain	Others	Refund Initiated	
2	1	Abhyuday	7	0.13	0	0	0	0	7	0	0	0	0	0	5	0	0	
3	2	Adarsh Co	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	
4	3	ADARSH M	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	
5	4	Airtel Payr	739	6.68	126	4.46	3	0	1260	0	10	0	0	26	663	151	0	
6	5	American	5	13.71	0	0	0	0	0	0	0	0	0	0	5	1	0	
7	6	Amreli Jili	1	0.16	0	0	0	0	0	0	0	0	0	0	1	0	0	
8	7	ANDAMAN	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
9	8	Andhra Pr	9	1.98	2	0	1	0	10	0	2	0	0	0	8	0	0	
10	9	Andhra Pr	17	1.14	0	0	2	0	18	0	0	0	0	0	17	0	0	
11	10	Aryavart B	4	0	0	0	4	0	8	0	0	0	0	0	3	0	0	
12	11	Assam Gr	6	0	0	0	6	0	6	0	0	0	0	0	5	0	0	
13	12	ASSOCIATI	1	0	0	0	0	0	2	0	0	0	0	0	1	0	0	
14	13	AU Bank	178	5.19	48	6.73	0	0	218	0	21	0	3	6	165	15	0	
15	14	Axis Bank	1518	175.29	321	26.19	239	0	2436	0	127	0	8	1	1411	56	0	
16	15	Bandhan I	339	28.86	41	1.44	8	0	461	0	69	4	8	0	311	52	0	
17	16	Bank of Be	18	5.8	0	0	3	0	2	0	0	0	0	0	16	2	0	
18	17	Bank of Be	1308	89.49	1115	32.11	98	0	1848	0	306	0	19	17	1216	262	0	
19	18	Bank of In	673	44.16	309	11.23	26	0	950	0	145	8	11	11	602	44	0	
20	19	Bank of M	541	13.69	207	10.54	136	0	838	0	206	0	0	0	508	3	0	
21	20	Baroda Gu	6	0.31	3	0.51	0	0	5	0	3	0	0	0	6	4	0	
22	21	Baroda Re	87	0.22	1	0.03	25	0	121	0	2	0	1	3	76	4	0	
23	22	Baroda Uf	39	1.43	0	0	7	0	37	0	0	0	0	0	36	0	0	
24	23	Bharat Co	3	0	2	0.09	0	0	3	0	0	0	0	0	3	0	0	
25	24	Bhatpara	1	0.05	0	0	1	0	0	0	0	0	0	0	1	0	0	

○○○

```
file_path = "Status Wise Repor 09_02_2025 22_06_11.csv"
date_match = re.search(r'\d{2}_\d{2}_\d{4}', file_path)
report_date = date_match.group().replace("_", "/") if date_match else "Unknown Date"
```

- Extracts the **date from the file name** (e.g., 09_02_2025 → 09/02/2025).
- This date is later used for **report title and file naming**.

Step 3: Load CSV Data and Clean Column Names

○○○

```
df_original = pd.read_csv(file_path)
df_original.columns = df_original.columns.str.strip()
```

- Reads the **CSV file** into a Pandas DataFrame.
- **Removes any leading/trailing spaces** in column names.

Step 4: Extract Relevant Columns

○○○

```
df = df_original[['Bank/Wallet/Merchant', 'Total Complaint', 'Complaint Un attended']].copy()
```

- Extracts only necessary columns:
 - 'Bank/Wallet/Merchant' → Bank names.
 - 'Total Complaint' → Total registered complaints.
 - 'Complaint Un attended' → Complaints that remain unresolved.

Step 5: Rename Columns for Readability

〇〇〇

```
df.rename(columns={
    'Bank/Wallet/Merchant': 'Bank',
    'Total Complaint': 'Total Complaint Reported',
    'Complaint Un attended': 'Unattended'
}, inplace=True)
```

- Renames columns for better readability.

Step 6: Calculate 'Unattended %' Column

〇〇〇

```
df['Unattended %'] = ((df['Unattended'] / df['Total Complaint Reported']) * 100).round(2).astype(str) + '%'
```

- Computes the percentage of unattended complaints.
- Rounds to two decimal places and appends %.

Step 7: Apply Filters

Filter 1: Remove Banks with Low 'Unattended %'

〇〇〇

```
df['Unattended %'] = df['Unattended %'].str.rstrip('%').astype(float)
df = df[df['Unattended %'] >= 10]
```

- Converts 'Unattended %' back to float for filtering.
- Removes banks where unattended complaints are less than 10%.

Filter 2: Remove Invalid Bank Names

〇〇〇

```
df = df[~df['Bank'].isin(['-----', '', 'Others'])]
```

- Removes invalid bank names (placeholders, blanks, and 'Others').

Filter 3: Select Exactly 50 Banks

〇〇〇

```
df = df.nlargest(50, 'Unattended')
```

- Selects the top 50 banks with the highest unattended complaints.

Step 8: Fix Serial Numbering (S. No.)

○ ○ ○

```
df.reset_index(drop=True, inplace=True)
df.insert(0, 'S. No.', range(1, len(df) + 1))
```

- Resets the index to avoid gaps.
- Adds a 'S. No.' column starting from 1 to 50.

Step 9: Save Data to Excel with Formatting

➔ Define File & Sheet Names

○ ○ ○

```
output_filename = f"Bankwise_Complaints_{report_date.replace('/', '_')}.xlsx"
sheet_name_processed = f"Processed_{report_date.replace('/', '_')}"
sheet_name_original = f"Original_{report_date.replace('/', '_')}"
```

- Generates dynamic filenames based on the extracted date.

➔ Write Data to Excel

○ ○ ○

```
with pd.ExcelWriter(output_filename, engine="openpyxl") as writer:
    df_original.to_excel(writer, sheet_name=sheet_name_original, index=False)
    df.to_excel(writer, sheet_name=sheet_name_processed, index=False, startrow=2, header=False)
```

- Stores original data in one worksheet.
- Saves processed data in another worksheet.
- Disables automatic header writing to prevent duplicate headers.

Step 10: Apply Formatting in Excel

➔ Add a Title Row

○ ○ ○

```
title = f"BANKWISE COMPLAINTS PENDENCY REPORT ON NCRP ({report_date})"
worksheet.merge_cells(start_row=1, start_column=1, end_row=1, end_column=len(df.columns))
title_cell = worksheet.cell(row=1, column=1)
title_cell.value = title
title_cell.font = Font(bold=True, size=12)
title_cell.alignment = Alignment(horizontal="center", vertical="center")
```

- Adds a merged title in bold & centered text.

➔ Manually Insert Headers in Row 2

○ ○ ○

```
for col_num, column_title in enumerate(df.columns, start=1):
    worksheet.cell(row=2, column=col_num, value=column_title).font = Font(bold=True)
```

- Ensures headers appear only once in Row 2.

➔ Auto-Adjust Column Width

○ ○ ○

```
for col_num, column_title in enumerate(df.columns, start=1):
    column_letter = get_column_letter(col_num)
    worksheet.column_dimensions[column_letter].width = max_length + 2
```

- Adjusts column width dynamically.

➔ Bold Bank Names

○ ○ ○

```
for row in range(3, len(df) + 3):
    worksheet.cell(row=row, column=2).font = Font(bold=True)
```

- Boldens bank names for better readability.

Final Outcome

	A	B	C	D	E	F
1		BANKWISE COMPLAINTS PENDENCY REPORT ON NCRP (09/02/2025)				
2	S. No.	Bank	Total Complaint Reported	Unattended	Unattended %	
3	1	IDBI Bank	507	250	49.31%	
4	2	Indian Overseas Bank	482	246	51.04%	
5	3	Kotak Mahindra Bank	1114	246	22.08%	
6	4	Axis Bank	1518	239	15.74%	
7	5	Union Bank of India(including Andhra Bank and Corporation Bank)	872	194	22.25%	
8	6	Bank of Maharashtra	541	136	25.14%	
9	7	Central Bank of India	394	102	25.89%	
10	8	Indian Bank (including Allahabad Bank)	564	82	14.54%	
11	9	Ratnakar Bank Limited (RBL)	331	73	22.05%	
12	10	Utkarsh Small Finance Bank Limited	166	68	40.96%	
13	11	India Post Payments Bank	483	62	12.84%	
14	12	Kerala Gramin Bank	75	51	68.0%	
15	13	The Gayatri Co-operative Urban Bank Ltd	43	43	100.0%	
16	14	Jio Payments Bank Ltd	219	39	17.81%	
17	15	Jammu and Kashmir Bank	138	31	22.46%	
18	16	Punjab & Sind Bank	221	30	13.57%	
19	17	Baroda Rajasthan Kshetriya Gramin Bank	87	25	28.74%	
20	18	PRAGATHI KRISHNA GRAMIN BANK(DIRECT)	16	16	100.0%	
21	19	Punjab Gramin Bank	18	14	77.78%	
22	20	Prathama UP Gramin Bank	17	13	76.47%	
23	21	NORTH EAST SMALL FINANCE BANK	22	10	45.45%	
24	22	KARNATAKA VIKAS GRAMEENA BANK	10	9	90.0%	
25	23	Madhya Bihar Gramin Bank	9	9	100.0%	
26	24	TELANGANA STATE CO-OP APEX BANK(SPO-IDBI)	9	9	100.0%	
27	25	Coastal Local Area Bank	8	8	100.0%	

Conclusion

This project successfully automates the processing and formatting of bank complaints data into a well-structured Excel report. The script ensures **data accuracy, proper filtering, and professional formatting**, making it highly efficient for analyzing unattended complaints trends.

KEY EFFICIENCY GAINS WITH FIRA

1. Reducing Manual Processing by 90%+

- FIRA automates the extraction, validation, and processing of financial fraud data from multiple sources, eliminating the need for manual data entry.
- This allows I4C officers to focus on analysis and decision-making rather than repetitive data processing tasks.

2. Accelerating Fraud Detection & Reporting by 80%

- The automation system can scan, clean, and validate thousands of financial complaint reports in seconds.
- It filters out irrelevant or duplicate cases, ensuring that I4C focuses on high-priority fraud incidents.

3. Enhancing Accuracy by 99.9%

- Standardized Data Processing:
 - Fixes misspellings and ensures uniform district and bank names.
 - Removes junk values and inconsistencies that often lead to reporting errors.
- Automated Verification:
 - Validates IFSC codes, account numbers, and transaction details against official databases.

4. Automating Compliance & Data Standardization

- District-Level Aggregation:
 - Fraud data is mapped to the correct districts, helping I4C identify cybercrime hotspots.
- Uniform Data Formatting:
 - Ensures compliance with regulatory bodies such as the RBI, NPCI, and banking institutions.
- Structured Reports:
 - FIRA generates structured, ready-to-use reports for seamless collaboration between law enforcement agencies and financial institutions.

5. Reducing Response Time by 70%

- Automated Complaint Analysis:
 - Flags unattended complaints and categorizes them based on severity, reducing investigation delays.

HOW FIRA HELPS I4C IN CYBERCRIME INVESTIGATIONS?

➔ **Before FIRA:**

- Officers manually review thousands of fraud reports, leading to delays and errors.
- Mismatched or inconsistent data makes analysis difficult.
- Cybercrime hotspots remain undetected due to unstructured data.

➔ **With FIRA:**

- Automated data structuring and fraud detection enables quick case resolution.
- Standardized reports help identify emerging fraud patterns in real time.
- Law enforcement receives clean, actionable insights within minutes.

CONCLUSION

FIRA is not just an automation tool; it is an intelligence-driven fraud prevention system. By integrating **AI-powered automation** with financial fraud analytics, FIRA can transform how I4C detects, analyzes, and responds to cybercrime in India. Faster, smarter, and error-free investigations will enhance national cybersecurity efforts, ensuring a safer digital ecosystem for financial transactions.

The internship at the **Indian Cyber Crime Coordination Centre (I4C), Cyber Fraud Mitigation Centre (CFMC)** provided valuable hands-on experience in **cyber fraud detection, financial crime analysis, and fraud mitigation strategies**. Through a combination of **data-driven insights, automation, and real-time fraud monitoring**, this work contributed to enhancing the operational effectiveness of fraud prevention mechanisms.

One of the key contributions was the **enhancement of the Cyber Crime Dashboard**, making it more efficient in tracking fraud patterns and providing **actionable intelligence to law enforcement agencies**. Additionally, the **multilingual public awareness campaign**, which included **caller tunes in various Indian languages**, aimed to educate citizens and reduce the likelihood of fraud victimization. The development of **scripts for geographic data extraction** ensured accurate **district and state identification**, which is crucial for **mapping cybercrime trends** and assisting law enforcement in targeted fraud mitigation efforts.

Another major aspect of this work involved **news analysis using the News API**, which allowed for **real-time tracking of emerging cyber fraud trends and scams**. Additionally, The automation of **data cleaning for IFSC codes and financial fraud reports** further improved **data accuracy and reliability**, enabling more **effective fraud detection**.

This internship reaffirmed the **importance of automation, real-time monitoring, and inter-agency collaboration** in the fight against cyber fraud. The findings highlight the need for **continuous innovation in fraud mitigation, strengthened coordination between financial institutions and cybercrime agencies, and the adoption of advanced cybersecurity measures**. Moving forward, these efforts will play a crucial role in **enhancing India's cybersecurity framework, protecting individuals and businesses from financial fraud, and strengthening the overall digital security ecosystem**.

REFERENCES

1. Indian Cyber Crime Coordination Centre (I4C) - Official resources and frameworks for cyber fraud mitigation.
2. Ministry of Home Affairs (MHA), Government of India - Guidelines and policies on cybercrime prevention.
3. National Cyber Crime Reporting Portal (NCRP) - Reporting mechanisms and fraud analysis methodologies.
4. Reserve Bank of India (RBI) Guidelines - Regulatory frameworks on banking fraud reporting and financial security.
5. Python Documentation (docs.python.org) - References for scripting techniques used in data processing and automation.
6. Python Libraries documentation: pandas, openpyxl, Matplotlib, RapidFuzz
7. Google Colab (colab.research.google.com) - Environment used for executing scripts related to data extraction and analysis.
8. Cybersecurity and Infrastructure Security Agency (CISA) - Best practices for fraud prevention and financial crime detection.
9. Financial Action Task Force (FATF) Reports - Global standards on anti-money laundering (AML) and combating financial fraud.
10. Research Papers on Cyber Fraud Mitigation - Various academic and industry sources discussing trends in cyber fraud, financial security, and fraud analytics.